IBM Db2 TechX Summit 2026 · AI / RAG

YojanaLens

Db2 decides eligibility. The model explains it.

An auditable welfare-scheme eligibility agent for Indian citizens and the caseworkers who help them. Deterministic SQL rules, native vector retrieval, and a stored decision trace behind every answer.

IBM Db2 12.1.5 | LangGraph | VECTOR(384, FLOAT32) | ibm_db | FastAPI

Solo submission · Aditya Lobhe · github.com/Aditya02-cpu

THE PROBLEM

Eligibility is written as prose, and the people who need it cannot read it

Citizens

Thousands of schemes across ministries and states. A widow in Uttar Pradesh may qualify for four at once, and typically learns about one.

Caseworkers

CSC operators and panchayat staff decide from memory. They cannot show the clause, so they cannot defend the answer.

Auditors

A decision made six months ago lives in a conversation, not in a record. There is nothing to replay.

A generic LLM makes this worse: it invents verdicts, cites clauses that do not exist, and leaves no trace. In welfare, an unauditable “you qualify” is a liability.

THE DESIGN DECISION

Split the job: the database decides, the model explains

Most AI demos

```
PDF → LLM → verdict → hope
```

- non-deterministic: same person, two answers
- cites clauses that were never retrieved
- no record of why the answer happened
- the model is both judge and narrator

YojanaLens

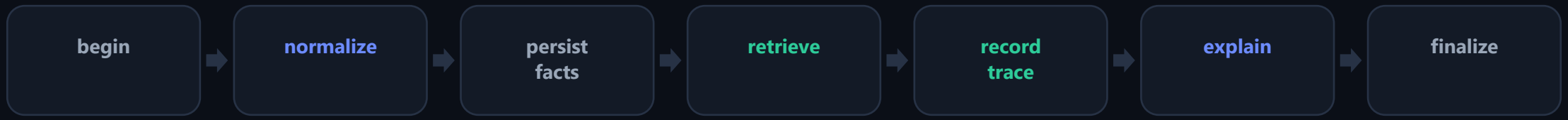
```
text → facts → Db2 SQL → verdict
                              ↓
                       LLM → explanation
```

- the verdict exists before the model runs
- the model receives it read-only
- every citation is checked against retrieval

"The model never sees a decision it is allowed to change."

ARCHITECTURE

One LangGraph run, with exactly one branch that matters



fastembed · ONNX

all-MiniLM-L6-v2
384 dims, offline, in the image

IBM Db2 12.1.5 · YOJANADB

8 tables · VECTOR(384, FLOAT32)
SP_RECORD_TRACE · 44 rules

LLM · OpenAI-compatible

narrates only
empty key → mock mode

`retrieve → Db2 vector search` `record_trace → Db2 rule matrix` `explain → LLM, citations validated`

8 nodes · conditional edge: no facts extracted → clarify, and the run is still audited

INSIDE DB2

The decision is a SQL statement, not a prompt

schemes	15 active · level, state, category
scheme_rules	44 criteria · eq / in / lte / gte
scheme_chunks	39 clauses · native VECTOR(384)
profile_facts	normalised applicant facts (EAV)
decision_trace	1 row per criterion, per run
audit_log	append-only event log

SP_RECORD_TRACE · sql/04_procedure.sql

```
CASE
  WHEN f.fact_key IS NULL THEN 'UNKNOWN'
  WHEN r.rule_op = 'eq' THEN
    CASE WHEN LOWER(f.fact_text) =
              LOWER(r.value_text)
         THEN 'PASS' ELSE 'FAIL' END
  WHEN r.rule_op = 'in' THEN
    CASE WHEN LOCATE(','||LOWER(f.fact_text)||',',
                     ','||LOWER(r.value_text)||',') > 0
         THEN 'PASS' ELSE 'FAIL' END
  WHEN r.rule_op = 'lte' THEN
    CASE WHEN f.fact_num <= r.value_num
         THEN 'PASS' ELSE 'FAIL' END
  WHEN r.rule_op = 'gte' THEN
    CASE WHEN f.fact_num >= r.value_num
         THEN 'PASS' ELSE 'FAIL' END
END
```

Python passes two strings: a run id and a profile id.

THE DB2 ARGUMENT

Retrieval and decision share one engine, one transaction

1 · Retrieval

Residency filter + cosine ranking, one round trip

```
WITH candidate AS (
  SELECT s.scheme_id FROM schemes s
  WHERE s.status = 'ACTIVE'
    AND s.state_code IN ('ALL', ?) )
SELECT c.chunk_id, VECTOR_DISTANCE(
  c.embedding,
  VECTOR(CAST(? AS VARCHAR(32672)),
         384, FLOAT32),
  COSINE) AS distance
FROM scheme_chunks c
ORDER BY distance FETCH FIRST 12 ROWS ONLY
```

2 · Decision

Every criterion of every active scheme, once

```
CALL SP_RECORD_TRACE(?, ?)

-- inside the procedure:
INSERT INTO decision_trace
SELECT ... , CASE ... END
FROM scheme_rules r
JOIN schemes s  ON s.scheme_id = r.scheme_id
LEFT JOIN profile_facts f
  ON f.fact_key = r.criterion_code
 AND f.profile_id = ?
```

No vector store to keep in sync. The clause, the rule and the verdict are one join.

DECISION POLICY

A verdict is not one bit

ELIGIBLE	No mandatory failure, no missing mandatory fact	PM-KISAN: farmer, 2 acres, no income tax
NEAR MISS	Exactly one mandatory failure, and it is numeric	Income 1.8L vs 1.5L cap — a door that can open
NEEDS INFO	No failure, but a mandatory fact is unknown	Ujjwala needs BPL status; a ration card is not proof
NOT ELIGIBLE	Categorical mismatch, or more than one failure	SC/ST scholarship for an OBC household

Conflating a threshold with a categorical mismatch would produce false hope. That is the failure mode this project exists to avoid.

DEMO HOUSEHOLD

One profile, four verdicts, one scheme filtered before scoring

THE INPUT

- 34-year-old widow
- OBC household
- Rural Uttar Pradesh
- Farms 2 acres
- Kutcha house
- ₹1.8 lakh a year
- Daughter in class 12
- Bank account

ELIGIBLE 6	NEAR MISS 3	NEEDS INFO 1	NOT ELIGIBLE 4
PM-KISAN	Sukanya Samriddhi	Ujjwala 2.0	Post-matric SC/ST
PM-JAY	Widow pension		PM SVANidhi
PMAY-Gramin	NFSA priority card		PMKVY 4.0
Kanya Sumangala			PM Vishwakarma
Atal Pension			
Fasal Bima			

+1 state scheme (Madhya Pradesh) excluded by the residency pre-filter in SQL · pinned by tests/test_corpus.py

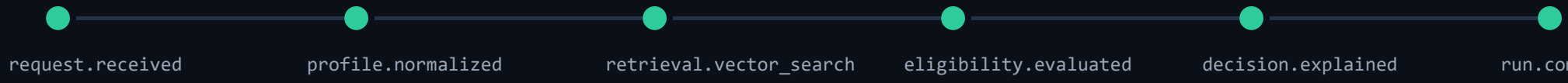
GUARDRAILS

Five guarantees that are structural, not prompt suggestions

1	**Verdicts are written before the model runs**	decision_trace is populated by SP_RECORD_TRACE; narration is an UPDATE on rows that already exist.
2	**Citations are validated**	Any chunk id the model returns that was not retrieved for that scheme is dropped.
3	**Unknown stays unknown**	A missing mandatory fact yields CLARIFY with a question. A ration card does not imply BPL status.
4	**The demo cannot die on a quota**	Empty API key, provider outage, or malformed JSON fall back to templated narration. Same verdicts.
5	**Closed fact vocabulary**	Facts are coerced against an allow-list before SQL. An invented field never reaches the database.

AUDITABILITY

Every answer is replayable from the database



decision_trace

one row per criterion, per run

`run_id` · `scheme_id` · `rule_id` · `criterion` · `observed value` · `verdict` · `clause_ref` · `narrative`

agent_runs

`model` · `prompt version` · `candidates` · `latency` · `summary JSON`

GET /api/runs/{id} returns the run, every rule row and the event log — no LLM in the loop

VERIFICATION

54 tests, no database required

- ✓ Corpus pins the exact demo verdicts (6 / 3 / 1 / 4)
- ✓ The real LangGraph runs end to end against a stubbed Db2 layer
- ✓ Near-miss vs categorical rejection is unit tested
- ✓ “My daughter is in class 12” never becomes the applicant's education
- ✓ Invented citations are dropped; provider failure degrades to mock
- ✓ SQL assets and parameter markers cannot drift apart
- ✓ The graph compiles and a missing driver explains the Docker path

reproduce

```
$ docker compose up --build
$ docker compose exec app \
    python -m pytest tests -q
54 passed

$ docker compose exec db2 su - db2inst1 -c \
    "db2 connect to YOJANADB && \
     db2 'select count(*) from schemes'"
15
```

Unit tests run without Db2. The container is the supported runtime for the full path.

HONEST BOUNDARIES

What it does not do yet

- Brute-force vector scan over 39 clauses — right at this size, documented
- Curated corpus, not a live feed; every scheme links to its official source
- English embeddings; queries are normalised before embedding
- Closed rule vocabulary — 15 criteria, extended deliberately
- No auth or multi-tenancy; profiles are personal data

NEXT

→ Db2 vector index as the corpus grows

→ State-specific rule packs

→ OCR intake for ration and land documents

→ Hindi UI, Granite narrator

→ Caseworker review queue

Db2 decides. The model explains. The trace is the evidence.

YojanaLens · submissions/yojana-lens · github.com/Aditya02-cpu